

Curso: Ciência da Computação

Nomes dos Participantes:

Henrique Floriano Alcantara – RM 575651

Katia Regina Bispo – RM 576044

João Pedro Candido Souza – RM 575884

Nicolas Klai de de França – RM 575343

Yago Souza Araujo – RM 575200

Relatório Operacional de Pré-Decolagem

Missão Aurora Siger - Fase 1

Este documento consolida as verificações necessárias para autorizar ou abortar o lançamento da nave Aurora Siger, bem como as decisões de engenharia, os algoritmos e as reflexões éticas e sustentáveis do projeto.

1.1 Organização e descrição da telemetria

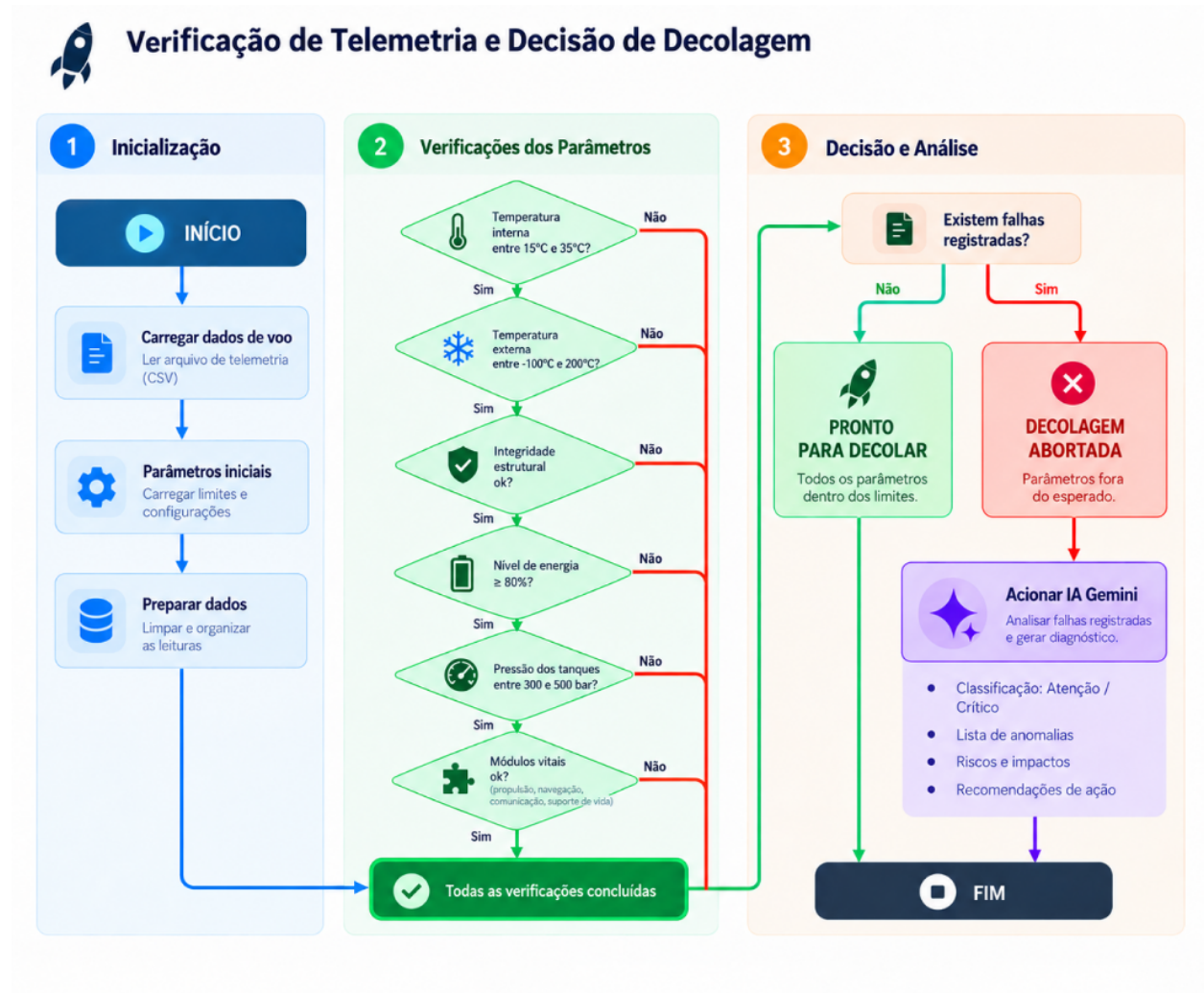
A telemetria da nave Aurora Siger é monitorada de forma contínua, processando sinais vitais para garantir a segurança no pré-lançamento. Os parâmetros e seus limites operacionais seguros são:

- **Temperatura Interna:** 15°C a 35°C (proteção de hardware e processadores).
- **Temperatura Externa:** -100°C a 200°C (resistência térmica estrutural do escudo).
- **Integridade Estrutural:** Exigido 1 (0 indica microfissuras ou colapso estrutural).
- **Nível de Energia:** Mínimo de 80% (garantia de autonomia para picos de ignição).
- **Pressão dos Tanques:** 300 kPa a 500 kPa (vazão correta de propelente).
- **Status dos Módulos Críticos:** Exigido 1 (subsistemas vitais 100% operacionais).

1.2 Algoritmo de verificação

O algoritmo de verificação atua através de desvios condicionais que avaliam os parâmetros capturados contra seus limites seguros de forma independente. Se todos os parâmetros estiverem dentro da faixa, o status final emitido é "PRONTO PARA

DECOLAR". Caso contrário, o lançamento é imediatamente classificado como "DECOLAGEM ABORTADA", e as falhas simultâneas são acumuladas para diagnóstico. A representação visual e a lógica passo a passo desta arquitetura podem ser conferidas abaixo:



1.3 Script em Python

O ecossistema embarcado da nave foi simulado computacionalmente através de um script na linguagem Python. Para garantir precisão de processamento autônomo, utilizou-se a biblioteca *Pandas* na extração otimizada dos dados a partir de um dataset em formato .csv (evitando preenchimentos manuais irreais). O script percorre os registros de telemetria aplicando as regras do algoritmo de verificação e exibe os diagnósticos no terminal em tempo real.

Código Python:

```
# Biblioteca do Gemini

!pip install -q google-genai

import pandas as pd

import time

from google import genai

from google.colab import files

from getpass import getpass


# 1. Pede a chave no momento da execução (mais seguro e fácil)

GEMINI_API_KEY = getpass("Cole sua Gemini API Key: ")


# 2. Configura o cliente passando a chave explicitamente

client = genai.Client(api_key=GEMINI_API_KEY)


# Gemini


def analyze_with_gemini(data):

    prompt = f"""

    Você é o sistema analítico de suporte à missão da nave Aurora Siger.

    Analise esta leitura de telemetria que apresentou falha no pré-lançamento:


    - Temperatura Interna: {data['internal_temperature']} °C (Ideal: 15 a 35)

    - Temperatura Externa: {data['external_temperature']} °C (Ideal: -100 a 200)

    - Integridade Estrutural: {data['structural_integrity']} (Exigido: 1)
```

FIAP – ATIVIDADE – FASE 1

- Nível de Energia: {data['energy_level']}% (Mínimo: 80)
- Pressão dos Tanques: {data['tank_pressure']} kPa (Ideal: 300 a 500)
- Status dos Módulos Críticos: {data['module_status']} (Exigido: 1)

Forneça uma resposta objetiva estruturada exatamente nos seguintes tópicos:

CLASSIFICAÇÃO: (ATENÇÃO ou CRÍTICO)

ANOMALIAS: (Descreva o que causou o problema)

RISCOS: (Impacto estrutural ou operacional)

RECOMENDAÇÃO: (Ajuste corretivo de engenharia)

"""

```
response = client.models.generate_content(  
    model="gemini-3.5-flash-lite",  
    contents=prompt  
)
```

```
return response.text
```

```
uploaded = files.upload()
```

```
print('Carregando telemetria da nave Aurora Siger\n')
```

```
df = pd.read_csv("telemetry_aurora.csv")
```

FIAP – ATIVIDADE – FASE 1

```
# Dados de energia

total_capacity = 150

estimated_consumption = 40

efficiency = 0.90


# Consumo total

total_energy_consumed = estimated_consumption / efficiency


# Perdas

energy_losses = total_energy_consumed - estimated_consumption


print('Arquivo carregado com sucesso!')

print(f'Total de registros: {len(df)}')


resultados = []


for index, row in df.iterrows():

    print(f'--- Analisando Leitura #{index + 1} ---')

    internal_temperature = row['internal_temperature']

    external_temperature = row['external_temperature']

    structural_integrity = row['structural_integrity']

    energy_level = row['energy_level']

    tank_pressure = row['tank_pressure']
```

FIAP – ATIVIDADE – FASE 1

```
module_status = row['module_status']

# Energia

available_energy = (total_capacity * energy_level) / 100

remaining_energy = available_energy - total_energy_consumed

print(f'Energia Disponível: {available_energy:.2f} kWh')

print(f'Consumo estimado (com rendimento): {total_energy_consumed:.2f} kWh')

print(f'Perdas energéticas: {energy_losses:.2f} kWh')

print(f'Autonomia restante estimada: {remaining_energy:.2f} kWh\n')

safe_takeoff = True

failures = []

# Parâmetros

parameters = {

    "Temperatura Interna": 15 <= internal_temperature <= 35,

    "Temperatura Externa": -100 <= external_temperature <= 200,

    "Integridade Estrutural": structural_integrity == 1,

    "Nível de Energia": energy_level >= 80,

    "Pressão dos Tanques": 300 <= tank_pressure <= 500,

    "Status dos Módulos Críticos": module_status == 1

}

# Verificação

for parameter_name, valid_status in parameters.items():
```

FIAP – ATIVIDADE – FASE 1

```
if not valid_status:

    safe_takeoff = False

    failures.append(parameter_name)


# Status

if safe_takeoff:

    status = "PRONTO PARA DECOLAR"

    print('STATUS: PRONTO PARA DECOLAR\n')

else:

    status = "DECOLAGEM ABORTADA"

    print('STATUS: DECOLAGEM ABORTADA.\n')

    print('Os Sistemas abaixo falharam:\n')


    for failure_reason in failures:

        print(f' - {failure_reason}')


    print()


# Salva o resultado

resultado = {

    "leitura": index + 1,

    "internal_temperature": internal_temperature,

    "external_temperature": external_temperature,

    "structural_integrity": structural_integrity,

    "energy_level": energy_level,

    "tank_pressure": tank_pressure,
```

FIAP – ATIVIDADE – FASE 1

```
"module_status": module_status,

"available_energy": available_energy,

"remaining_energy": remaining_energy,

"safe_takeoff": safe_takeoff,

"status": status,

"failures": failures

}

resultados.append(resultado)

print('-' * 70)

print()

print(f'Varredura completa dos {len(df)} registros finalizada com sucesso.')

numero_leitura = 1

resultado = resultados[numero_leitura - 1]

print(f'Leitura: {resultado["leitura"]}')

print(f'Temperatura interna: {resultado["internal_temperature"]} °C')

print(f'Temperatura externa: {resultado["external_temperature"]} °C')

print(f'Nível de energia: {resultado["energy_level"]}%')

print(f'Pressão dos tanques: {resultado["tank_pressure"]} kPa')

print(f'Energia disponível: {resultado["available_energy"]:.2f} kWh')
```


FIAP – ATIVIDADE – FASE 1

```
print(f'Autonomia restante: {resultado["remaining_energy"]:.2f} kWh')

print(f'Status: {resultado["status"]}')

print(f'Falhas: {resultado["failures"]}')


for resultado in resultados:

    if resultado["safe_takeoff"] == False:

        print(f'--- Gemini analisando Leitura #{resultado["leitura"]} ---\n')

        failure_data = {

            'internal_temperature': resultado['internal_temperature'],

            'external_temperature': resultado['external_temperature'],

            'structural_integrity': resultado['structural_integrity'],

            'energy_level': resultado['energy_level'],

            'tank_pressure': resultado['tank_pressure'],

            'module_status': resultado['module_status']

        }

        try:

            print('[IA] Acionando triagem assistida do Gemini...\n')

            ai_analysis = analyze_with_gemini(failure_data)

            # Salva a resposta
```

FIAP – ATIVIDADE – FASE 1

```
    resultado["analise_ia"] = ai_analysis

    print(ai_analysis)

except Exception as e:

    print(f'Aviso de conexão com a API: {e}')

print('\n' + '=' * 100 + '\n')

time.sleep(1)

numero_leitura = 1

resultado = resultados[numero_leitura - 1]

if "analise_ia" in resultado:

    print(resultado["analise_ia"])

else:

    print("Esta leitura ainda não possui uma análise do Gemini salva.")

tabela_resultados = pd.DataFrame(resultados)

tabela_resultados
```

Link GitHub:

https://github.com/YagoSDev/Missao_Aurora_Siger

1.4 Análise energética

A gestão termodinâmica é crucial. Para validar a autonomia energética da missão, foi considerado um banco de baterias com capacidade hipotética de 150 kWh. O pico útil exigido na decolagem foi estimado em 40 kWh e a eficiência do sistema em 90%. Assim, o cálculo modelado encontrou o consumo total bruto de 44,44 kWh, sendo os 4,44 kWh adicionais referentes a perdas térmicas e dissipativas. Em cada iteração, o sistema converte a porcentagem atual da bateria em kWh disponível, abate o consumo e verifica matematicamente a energia restante para pós-decolagem.

Detalhamento do Cálculo de Análise Energética

O cálculo foi dividido em duas fases: a definição das constantes físicas do motor/bateria e a matemática dinâmica aplicada a cada linha de telemetria.

Definição das Constantes Técnicas (Passo Fixo)

Antes de processar os dados, o sistema define as características do banco de baterias e do consumo de decolagem:

- **Capacidade Total (total_capacity): 150 kWh.**
 - *Motivo físico:* Representa um banco de baterias de íon-lítio de alta densidade projetado para sustentar computadores de bordo, telemetria de alta frequência e sistemas inerciais durante o pré-lançamento.
- **Consumo Útil Estimado (estimated_consumption): 40 kWh.**
 - *Motivo físico:* É o pico instantâneo de energia exigido para acionar os ignitores, pressurizar tanques e fechar válvulas principais no momento do empuxo.
- **Eficiência do Sistema (efficiency): 0.90 (90%).**

Cálculo de Perdas e Consumo Real (Termodinâmica)

Com base na Primeira Lei da Termodinâmica e no Efeito Joule, sabemos que circuitos dissipam calor. O código calcula essas perdas antes da decolagem:

- **Consumo Total Real:** Calculado pela divisão do consumo útil pela eficiência da nave ($40 / 0.90$). O resultado é de **44,44 kWh**.
- **Perdas Energéticas:** Subtraindo o consumo útil do consumo total real ($44,44 - 40$), encontramos as perdas. O resultado é de **4,44 kWh**, que representa o calor dissipado de forma irreversível nos condutores e conversores.

Cálculo Dinâmico da Autonomia (A cada leitura do CSV)

Dentro do laço de repetição, o sistema pega a porcentagem atual da bateria (energy_level) e aplica a fórmula final:

- **Energia Disponível** = $(\text{total_capacity} * \text{energy_level}) / 100$.
- **Autonomia Restante** = Energia Disponível - Consumo Total Real.

1.5 Análise assistida por IA

Como um diferencial na arquitetura, o software conta com a integração da API do Google Gemini (modelo gemini-3.5-flash-lite). A inteligência artificial é invocada de forma programática toda vez que a decolagem é abortada, recendo o conjunto anômalo de dados. O modelo atua no suporte, analisando e devolvendo, em tempo de execução, um laudo estruturado contendo a Classificação de risco (Atenção/Crítico), as causas das Anomalias, os Riscos operacionais e a Recomendação técnica direcionada à equipe de engenharia.

1.6 Reflexão Crítica

Ética e responsabilidade. Em sistemas críticos, a automação deve ser explicável, confiável e auditável. O Machine Learning e a Inteligência Artificial podem auxiliar na identificação de anomalias e na análise dos dados, mas as regras de segurança e a decisão final devem permanecer sob responsabilidade humana e institucional. Dessa forma, evita-se que uma previsão incorreta da IA coloque a missão, os equipamentos ou pessoas em risco.

Impacto social. A exploração espacial pode trazer benefícios importantes para a sociedade, contribuindo para avanços científicos, comunicação, navegação, previsão do tempo e monitoramento ambiental. Entretanto, essas atividades também exigem grandes investimentos financeiros, tecnológicos e energéticos. Por isso, é importante avaliar se os benefícios gerados justificam os recursos utilizados e como essas tecnologias podem contribuir para a sociedade.

Impacto ambiental e sobre os animais. Lançamentos espaciais podem causar impactos no meio ambiente, como emissão de gases, poluição sonora e geração de resíduos. O ruído intenso e as alterações provocadas nas regiões próximas às bases de lançamento também podem afetar animais, especialmente aves e outras espécies sensíveis ao som e às mudanças em seu habitat. Além disso, componentes descartados durante missões podem atingir oceanos ou outras áreas naturais. Por esse motivo, locais e métodos de lançamento devem ser planejados buscando reduzir danos à fauna, à flora e aos ecossistemas.

Sustentabilidade. A análise das perdas energéticas realizada no projeto contribui para identificar desperdícios e melhorar a eficiência do sistema. Em aplicações reais, a sustentabilidade também envolve reduzir o consumo de recursos, aumentar a vida útil dos equipamentos, utilizar fontes de energia mais limpas, reaproveitar componentes quando possível e realizar a destinação adequada de resíduos eletrônicos e materiais utilizados nas missões.

Uso responsável de IA. Neste projeto, a IA atua como uma ferramenta de apoio e não possui autoridade para liberar uma decolagem. As verificações determinísticas continuam sendo responsáveis por identificar condições que violem os limites de segurança. Essa separação é importante porque modelos de IA podem apresentar erros ou classificações imprecisas. Assim, a tecnologia complementa a análise sem substituir critérios objetivos de segurança e a supervisão humana.

Referências:

Google AI for Developers – Gemini API: <https://aistudio.google.com/api-keys>

Google AI Studio – API Keys: <https://ai.google.dev/gemini-api/docs?hl=pt-br>

Materiais disponibilizados pela FIAP para a Fase 1 de Ciência da Computação.